

Homework 2

[2026-1] 데이터사이언스를 위한 컴퓨팅 (001)

Due: 2026년 4월 28일 23:59

1. Bank Account Management System [50pts]

In this homework, you will implement a Bank Account Management System using Object-Oriented Programming principles. Implement the TODO sections in `bankaccount.hpp`.

Transaction Struct

The `Transaction` struct is used to log each operation (deposit or withdrawal) on a bank account. Every time a deposit or withdrawal is made, a new `Transaction` is added to the account's `transactionHistory`. Each `Transaction` contains:

- `deposit`: The amount of money deposited into the account (if the transaction is a deposit, otherwise 0).
- `withdraw`: The amount of money withdrawn from the account (if the transaction is a withdrawal, otherwise 0).
- `balance`: The balance of the account after the transaction.

1.1 BankAccount

The `BankAccount` class defines the core operations of a bank account. It manages the account balance (`balance`) and transaction history (`transactionHistory`), and provides fundamental functionalities such as deposit and withdrawal. Please note that the deposit and withdrawal amount must be greater than 0. Any transactions with a non-positive amount are ignored and not logged.

1.1.1. BankAccount Constructor

Implement a constructor for the `BankAccount` class that accepts an initial balance and stores it in the `balance` variable. This constructor should initialize the account with the given balance, which represents the starting amount of money in the account.

- Parameter:
 - `double initialBalance`: The initial balance of the account.
- Return: None.

1.1.2. deposit Method

Declare a method `deposit` in the `BankAccount` class. This method should accept an amount to deposit into the account and log the deposit in the `transactionHistory`. When this method is called, a `Transaction` is logged with the `deposit` field set to the amount, the `withdraw` field set to 0, and the `balance` updated to reflect the new balance after the deposit.

- Parameter:
 - `double amount`: The amount to deposit into the account.
- Return: None.

1.1.3. withdraw Method

Declare a method `withdraw` in the `BankAccount` class. This method should accept an amount to withdraw from the account and log the withdrawal in the `transactionHistory`. The transaction should record the withdrawn amount, 0 for the deposit, and the balance after the withdrawal. If the amount to withdraw exceeds the current balance, the withdrawal will fail. Even if the withdrawal fails, the transaction should still be logged in the `transactionHistory`. In this case, the `deposit` should be 0, the `withdraw` should be 0, and the `balance` should remain.

- Parameter:
 - `double amount`: The amount to withdraw from the account.
- Return: None.

1.1.4. getBalance Method

Implement the `getBalance` method, which returns the current balance of the account. This method should allow users to check the balance without modifying it.

- Parameter: None.
- Return:
 - `double`: The current balance of the account.

1.1.5. getTransactionHistory Method

Implement the `getTransactionHistory` method, which returns a reference to the vector containing the `transactionHistory` of the account. This method provides read-only access to all recorded deposits and withdrawals.

- Parameter: None.
- Return:
 - `const std::vector<Transaction>&`: A reference to the vector of `Transaction` objects, which store the deposit, withdrawal, and balance data.

1.2. SavingsAccount

Savings accounts typically earn interest on deposits.

1.2.1. SavingsAccount Constructor

Implement a constructor for the `SavingsAccount` class. This constructor should initialize the `BankAccount` base class with an initial balance and store the interest rate.

- Parameter:
 - `double initialBalance`: The initial balance of the savings account.
 - `double interestRate`: The interest rate applied to the account. It is a value between 0 and 0.5.
- Return: None.

1.2.2. deposit Method (Override for SavingsAccount)

Override the `deposit` method for the `SavingsAccount` class. The interest is applied preemptively, at the time of the deposit. The total deposit consists of the sum of the amount and the interest, where interest is calculated as `amount * interestRate`. The `Transaction` should be logged with the total deposit. The `withdraw` field is set to 0. Ensure the balance is updated to the new balance after the deposit.

- Parameter:
 - `double amount`: The amount to deposit into the savings account.
- Return: None.

1.2.3. Destructor for SavingsAccount

Implement a destructor for the `SavingsAccount` class. The destructor should print a message "SavingsAccount closed".

- Parameter: None.
- Return: None.

1.3. CheckingAccount

Checking accounts typically charge a transaction fee for each withdrawal. This fee will be deducted from the account balance whenever the money is withdrawn.

1.3.1. CheckingAccount Constructor

Implement a constructor for the `CheckingAccount` class. This constructor should initialize the `BankAccount` base class with an initial balance and store the transaction fee.

- Parameter:
 - `double initialBalance`: The initial balance of the checking account.
 - `double transactionFee`: The transaction fee charged for each withdrawal.
- Return: None.

1.3.2. withdraw Method (Override for CheckingAccount)

Override the `withdraw` method for the `CheckingAccount` class. This method should withdraw the specified amount from the account, plus the `transactionFee`. If the balance is insufficient to cover both the amount and the fee, no withdrawal should occur. When a withdrawal is successful, a `Transaction` is logged with the `withdraw` field set to the sum of the amount and the `transactionFee`. The `deposit` field is set to 0, and the balance is updated to reflect the new balance after the fee is applied. Even if the withdrawal fails, the transaction should still be logged in the `transactionHistory`. The `deposit` should be 0, the `withdraw` should be 0, and the balance should remain unchanged.

- Parameter:
 - `double amount`: The amount to withdraw from the checking account.
- Return: None.

1.3.3. Destructor for `CheckingAccount`

Implement a destructor for the `CheckingAccount` class. The destructor should print a message “`CheckingAccount` closed”.

- Parameter: None.
- Return: None.

1.4. `BusinessAccount`

Business accounts typically offer a credit line, which allows the account to withdraw funds even if the balance is insufficient, as long as the total withdrawal amount is within the credit limit. For instance, if the balance is 1000 and the credit line is 200, you can withdraw up to 1200, leaving the balance at -200. The credit line is greater than 0.

1.4.1. `BusinessAccount` Constructor

Implement a constructor for the `BusinessAccount` class. This constructor should initialize the `BankAccount` base class with an initial balance and store the credit line.

- Parameter:
 - `double initialBalance`: The initial balance of the business account.
 - `double creditLine`: The credit line available for the account.
- Return: None.

1.4.2. `withdraw` Method (Override for `BusinessAccount`)

Override the `withdraw` method for the `BusinessAccount` class. This method should allow withdrawals up to the combined total of the balance and the credit line. If the withdrawal amount exceeds the available balance plus the credit line, no withdrawal should occur. When a withdrawal is successful, a `Transaction` is logged with the `withdraw` field set to the amount. The `deposit` field is set to 0, and the balance is updated to reflect the new balance, using the available credit line if necessary. Even if the withdrawal fails, the transaction should still be logged in the `transactionHistory`. The `deposit` should be 0, the `withdraw` should be 0, and the

balance should remain unchanged.

- Parameter:
 - `double amount`: The amount to withdraw from the business account.
- Return: None.

1.4.3. Destructor for `BusinessAccount`

Implement a destructor for the `BusinessAccount` class. The destructor should print a message “`BusinessAccount` closed”.

- Parameter: None.
- Return: None.

1.5. AccountManager

The `AccountManager` is responsible for managing multiple bank accounts, allowing for operations such as adding new accounts and transferring funds between them.

1.5.1. `addAccount` Method

Implement the `addAccount` method in the `AccountManager` class. This method should add a new bank account to the manager's list of accounts. The method should store the account as a `std::shared_ptr<BankAccount>`.

- Parameter:
 - `const std::shared_ptr<BankAccount>& account`: A shared pointer to the account being added.
- Return: None.

1.5.2. `transferFunds` Method

Implement the `transferFunds` method in the `AccountManager` class. This method should transfer money from one account to another. The method should first withdraw the specified amount from the account at `fromIndex` and then deposit the same amount into the account at `toIndex`. If either index is invalid, throw a `runtime_error` with a message “Invalid account index”. Ensure that `transferFunds` can only be proceeded if the withdrawal from the account at `fromIndex` is successfully completed.

- Parameter:
 - `int fromIndex`: The index of the account to withdraw from.
 - `int toIndex`: The index of the account to deposit into.
 - `double amount`: The amount to transfer between accounts.
- Return: None.

1.5.3. `getAccount` Method

Implement the `getAccount` method in the `AccountManager` class. The `AccountManager` keeps track of all accounts it manages. This method provides access to specific accounts by their index in the account list. This method should return a shared pointer to the bank account at the specified index. If the index is invalid, throw a `runtime_error` with a message “Invalid account index”.

- Parameter:
 - `int index`: The index of the account to retrieve.
- Return:
 - `std::shared_ptr<BankAccount>`: A pointer to the account at the given index.

2. Custom Map [50pts]

This assignment involves implementing a map that is similar to `std::map` in the C++ standard library. A map typically consists of key-value pairs, and its construction allows for the specification of the types of keys and values through templates. Additionally, the template for the map takes `std::less` as the default type for `Compare` and `std::allocator` as the default type for `Allocator`. Figure 1 is the prototype of the actual C++ `std::map`.

```
template<
    class Key,
    class T,
    class Compare = std::less<Key>,
    class Allocator = std::allocator<std::pair<const Key, T>>
> class map;
```

Figure 1. Prototype of `std::map`

The map to be implemented in this assignment should be designed as a template that takes `K` as the data type for the key, `V` as the data type for the value, and `Compare` as a comparison class. In this assignment, you don't have to consider the `Allocator` type. `std::less` typically takes two data of the same type and compares which one is smaller. Below is an example usage of `std::less`. Additional explanations can be accessed through the [link](#).

```
template <typename K, typename V, typename Compare = std::less<K>>
class map {
private:
    // ....
    // other member variables
    // ....
    Compare comp;
    // ....
    // other member variables
    // ....
public:
    // ....
    // other member functions
    // ....
    void example_function(K key1, K key2){
        if(comp(key1,key2)){
            cout << "Key1 is smaller than key2" << endl;
        }else if(comp(key2,key1)){
            cout << "Key2 is smaller than key1" << endl;
        }
    }
};
```

Figure 2. Example of code for `std::less`

The objective of this assignment is to implement a `CustomMap` that stores data in a binary search tree (BST). A binary search tree stores data based on the key and the value is stored in the same `TreeNode` as a member variable. The goal is to implement operations such as look up, insert, and delete using `Compare` for comparing keys.

Instruction

Read all explanations for the sub-problems below and implement them in `custommap.hpp`. To receive full credit, you must declare all member functions and member variables as instructed in the subproblems. Additionally, declaring extra member functions and variables is permissible in the completion of the assignment.

Ensure your implementation of Binary Search Tree (BST) is correct, resulting in an **average time complexity of $O(\log N)$** . Rebalancing is not required. Be aware that **using `std::map` or `std::vector` is strictly prohibited. Including the `<map>` and `<vector>` headers will automatically result in 0 credits.**

2.1. `TreeNode` and `CustomMap` [10 pts]

Define the `TreeNode` class using a class template. The class template should take two template arguments: the type of the keys, the type of the values. `TreeNode` should declare private member variables `K key`, `V value`, `TreeNode* left`, and `TreeNode* right`. The constructor should take the `K key` and `V value` and assign them to private member variables accordingly. Additionally, there should be getter and setter functions for each member variable. The name of each getter, setter function is not fixed, so you can name it freely.

Similarly, define the `CustomMap` class using a class template. The class template should take three template arguments: the type of the keys, the type of the values, and the compare class `std::less<K>` for comparison inside the map data structure (refer to Figure 2). `CustomMap` class should have `TreeNode` corresponding to the root of the BST, `TreeNode* root`, and `Compare` object as member variables. The `CustomMap` constructor should not take any arguments. `CustomMap` class should have `Compare get_compare()` as a getter function.

2.2. `insert` [10 pts]

Write a `void insert(K key, V value)` function that inserts the key-value into the BST as a `TreeNode`, in accordance with the properties of a binary search tree. If the key exists in the tree, `insert` operation should be ignored. And also, write a function `void print_map()` that prints the key:value pairs stored within `CustomMap` in ascending order of keys. An example output can be seen in Figure 3. Make sure to print the “key:value\n” form exactly, without any space or other characters, or you won’t get any credits. Grading based on test cases rely on the output of the `print_map` function. So even if the `insert` function is correct, you will get no credits if the `print_map` function is not implemented correctly.

<pre>#include "custommap.hpp" int main() { CustomMap<int, std::string> map; map.insert(5, "Five"); map.insert(3, "Three"); map.insert(8, "Eight"); map.insert(10, "Ten"); map.print_map(); }</pre>	<pre>./main 3:Three 5:Five 8:Eight 10:Ten</pre>
---	---

Figure 3. Result of `insert` and `print_map`

2.3. `delete_key` [10 pts]

Write a function `void delete_key(K key)` that takes a key of type `K` as an argument and deletes the node corresponding to that key. If the input key does not exist, the delete function should do nothing.

2.4. `get_value` [10 pts]

Write a function `V get_value(K key)` that takes a key of type `K` as an argument and returns the corresponding value of type `V`. If the input key does not exist, the `get_value` function should return default value, `V()`. It should not insert a new element.

2.5. `[]` Operator [10 pts]

Overload the `[]` operator such that you can access a value using a key. When the key exists in the map, the operator returns the corresponding value. When the key does not exist, it inserts a new element with the key and the default value `V()`. Also, the operator should allow for modifying the corresponding value directly. For example, by `map[3] = 4`, this operator has to perform “adding a pair whose key is 3 and value is 4” if there is no key 3, and has to replace its value with 4 if the key exists.

Submission Instructions

- Complete the implementation of all TODOs in the `bankaccount.hpp` and `custommap.hpp`. Be careful with formatting and typos.
- For Homework 2 - Problem 1 on GradeScope, please submit `bankaccount.hpp`, and for Homework 2 - Problem 2, please submit `custommap.hpp`.
- Grading will be conducted using C++20 standard. The compilation process is as follows:

```
For both problems,  
$ g++ -c main.cpp -o main -std=c++20  
./main
```

- When submitting your codes, ensure that all test codes printing out `stdout` are removed, unless explicitly instructed otherwise. **Grading relies solely on the `stdout`, and including any unrelated codes may result in inaccurate grading.**
- You must **NOT** share your code with other students. Additionally, students must not rely on LLMs to generate code directly. Any student found to have a high level of code similarity, as determined by our similarity detection process, may face serious penalties.
- If you submit late, grace days will be automatically deducted. You have 5 grace days available for homework submissions throughout the semester. Grace days are counted in 24-hour increments from the original due time. For example, if an assignment is due on 1/1 at 11:59 PM, submitting it 30 minutes late will use one grace day, while submitting it on 1/3 at 9:00 PM will use two grace days. Late submissions will **NOT** be accepted once all grace days have been used.
- For questions about this homework, use the “Homework 2” discussion forum on eTL.
- Failing to adhere to the submission guidelines may result in penalties applied without exception.